

Pie Charts, Box Plots, Scatter Plots, and Bubble Plots

What I'm Doing

Using Matplotlib and pandas, I load and visualize Canadian immigration data (1980–2013) with pie, box, scatter, and bubble plots to highlight trends, compare distributions, and fit regression lines for predictions

1. Setting Up

First, I import the core libraries and set up Matplotlib with a clean visual style.

```
import numpy as np
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt

mpl.style.use('ggplot')
print('Matplotlib version:', mpl.__version__)
```

Matplotlib version: 3.10.9

Now I load the Canadian immigration dataset from the provided URL.

```
df_can = pd.read_csv(
    'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/
    IBMDeveloperSkillsNetwork-DV0101EN-SkillsNetwork/Data%20Files/Canada.csv')
```

2. Getting to Know the Dataset

Let's check what we're working with.

```
df_can.head()
```

	Country	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	...	2005	:
0	Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	...	3436	
1	Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	...	1223	
2	Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44	...	3626	
3	American Samoa	Oceania	Polynesia	Developing regions	0	1	0	0	0	0	...	0	
4	Andorra	Europe	Southern Europe	Developed regions	0	0	0	0	0	0	...	0	

5 rows × 39 columns

Let's find out how many entries there are in our dataset.

```
print(df_can.shape)
```

```
(195, 39)
```

The dataset has **195 countries** (rows) and **39 columns**

Setting the Country as Index

For quick lookups, I'll set the `Country` column as the index.

```
df_can.set_index('Country', inplace=True)
df_can.head()
```

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...	2005	:
Country													
Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	496	...	3436	
Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	1	...	1223	
Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44	69	...	3626	
American Samoa	Oceania	Polynesia	Developing regions	0	1	0	0	0	0	0	...	0	
Andorra	Europe	Southern Europe	Developed regions	0	0	0	0	0	0	2	...	0	

5 rows × 38 columns

Country names now serve as indices. I then check the dimension to confirm and create list of years for data visualization.

```
print('data dimensions:', df_can.shape)

years = list(map(str, range(1980, 2014)))

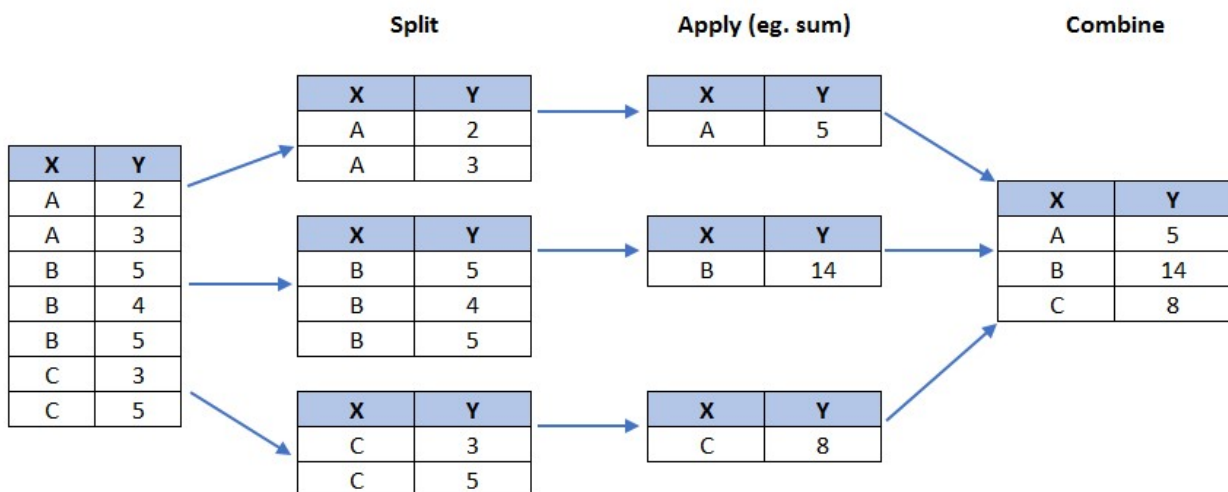
data dimensions: (195, 38)
```

3. Pie Charts

A **pie chart** displays **numeric proportions** by dividing a **circle into proportional slices**. It's a go-to visualization for showing parts of a whole.

I'll use a pie chart to explore the proportion of new immigrants grouped by continent across the entire 1980–2013 period.

Step 1: Gather Data with `groupby`



The `groupby` method splits the data into continent groups, applies a sum function, and combines the results. (another functions are: `.sum()` `.count()` `.mean()` `.std()` `.aggregate()` `.apply()` etc..)

```
df_continents = df_can.groupby('Continent', axis=0).sum()
df_continents.head()
```

		Region	DevName	1980	1981	1982	1983	198
Continent								
Africa	Northern Africa	Middle Africa	Developing regions	3951	4363	3819	2671	263
		Western Africa	Developing regions					
Asia	Southern Asia	Western Asia	Developing regions	31025	34314	30214	24696	2727
		Western Asia	Developing regions					
Europe	Southern Europe	Southern Europe	Developed regions	39760	44802	42720	24638	2228
		Western Europe	Developed regions					
Latin America and the Caribbean	America	Caribbean	Developing regions	13081	15215	16769	15427	1367
		Caribbean	Developing regions					
Northern America	Northern America	Northern America	Developed regions	9378	10030	9074	7100	666
			Developed regions					

5 rows × 37 columns

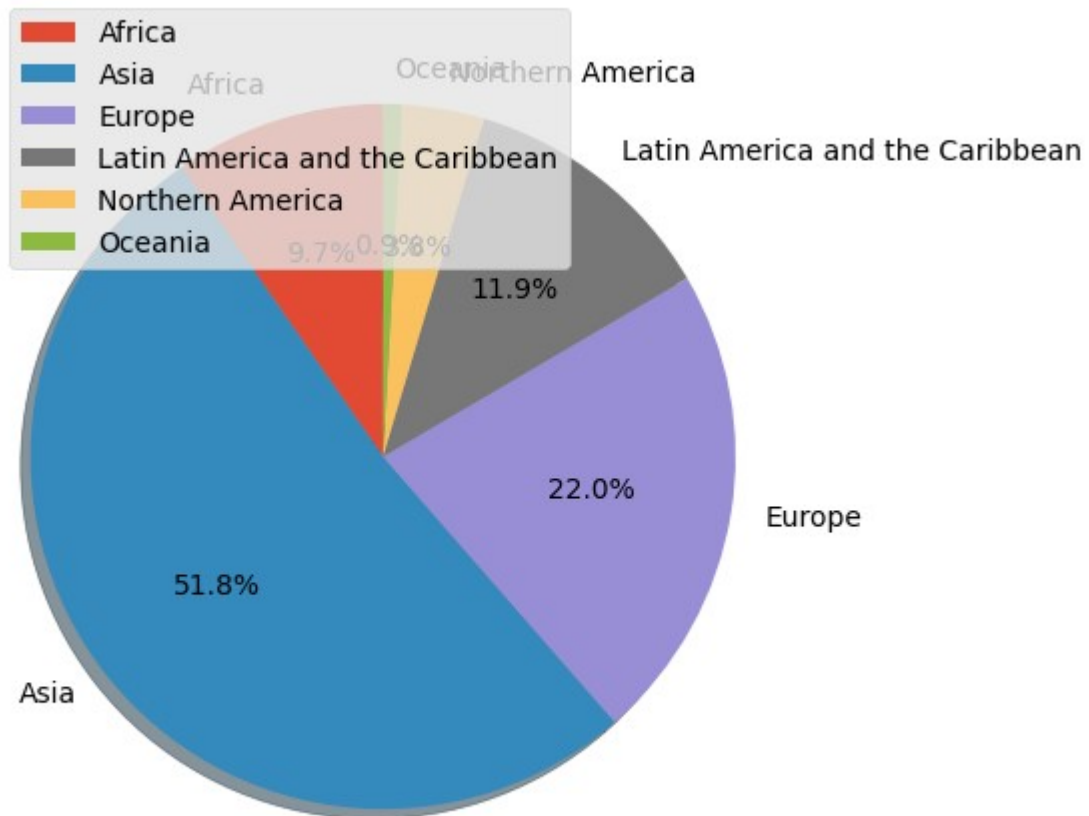
Step 2: Plot a Basic Pie Chart

- `autopct` - is a string or function used to label the wedges with their numeric value. The label will be placed inside the wedge. If it is a format string, the label will be `fmt%pct`.
- `startangle` - rotates the start of the pie chart by angle degrees counterclockwise from the x-axis.
- `shadow` - Draws a shadow beneath the pie (to give a 3D feel).

```
df_continents['Total'].plot(
    kind='pie',
    figsize=(5, 6),
    autopct='%1.1f%%',
    startangle=90,
    shadow=True,
)

plt.title('Immigration to Canada by Continent [1980 - 2013]')
plt.axis('equal')
plt.legend(labels=df_continents.index, loc='upper left')
plt.show()
```

Immigration to Canada by Continent [1980 - 2013]



Result: A pie chart appears, but the labels and percentages overlap in places. Time to clean it up.

Step 3: Refine the Pie Chart

I'll remove overlapping text labels, push percentages outward, add custom colors, and "explode" the smallest continents for emphasis.

```
colors_list = ['gold', 'yellowgreen', 'lightcoral', 'lightskyblue', 'lightgreen',  
'pink']
```

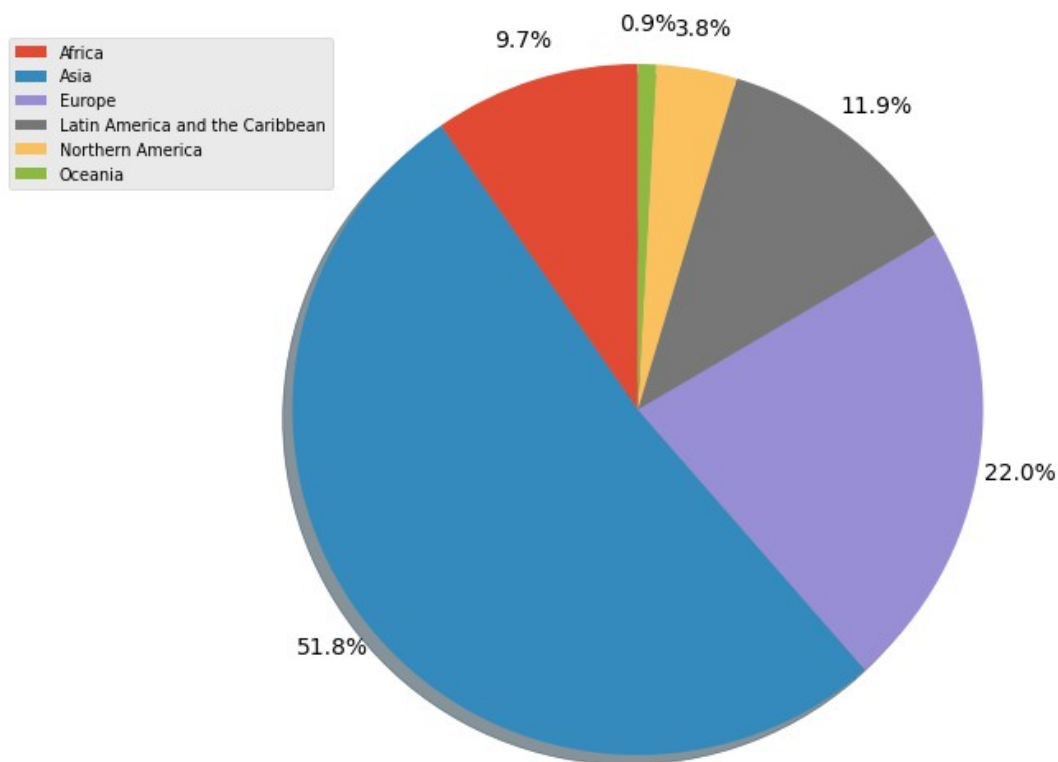
```
explode_list = [0.1, 0, 0, 0, 0.1, 0.1] # ratio for each continent with which to  
offset each wedge.
```

```
df_continents['Total'].plot(  
    kind='pie',  
    figsize=(10, 6),  
    autopct='%1.1f%%',  
    startangle=90,  
    shadow=True,  
    labels=None,  
    pctdistance=1.12,  
    # the ratio between the pie center and start of text label generated by  
    autopct  
    #colors=colors_list, # add custom colors  
    #explode=explode_list # 'explode' lowest 3 continents  
)
```

```
# scale the title up by 12% to match pctdistance
```

```
plt.title('Immigration to Canada by Continent [1980 - 2013]', y=1.12,
fontSize=15)
plt.axis('equal')
plt.legend(labels=df_continents.index, loc='upper left', fontsize=7)
plt.show()
```

Immigration to Canada by Continent [1980 - 2013]



Result: A clean, readable pie chart. The percentages sit outside the wedges, and a separate legend identifies each continent. **Europe** and **Asia** dominate, while **Oceania** and **North America** are the smallest slices.

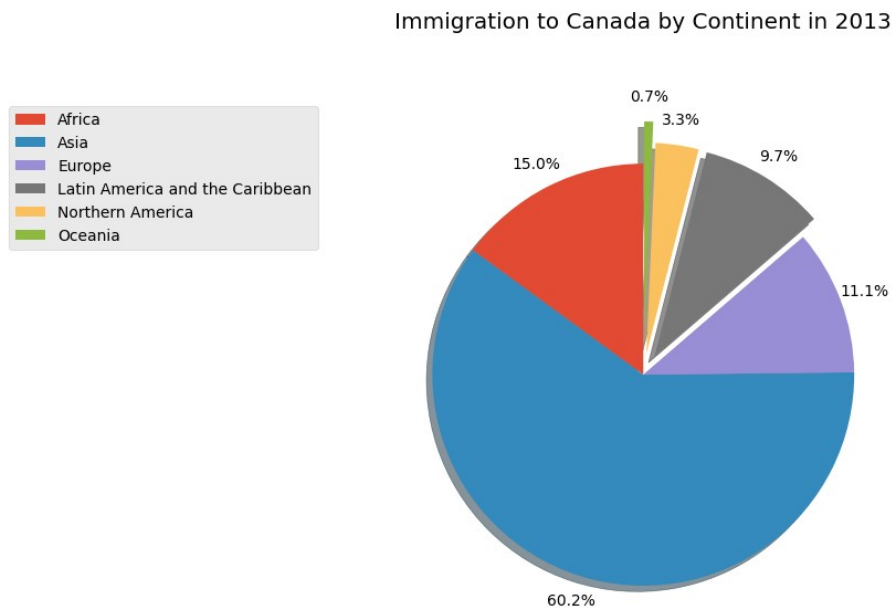
Exercise: Immigration by Continent in 2013

Goal: Create a pie chart showing the proportion of new immigrants grouped by continent, but only for the year **2013**.

```
explode_list = [0.0, 0, 0, 0.1, 0.1, 0.2]
```

```
df_continents['2013'].plot(
    kind='pie',
    figsize=(15, 6),
    autopct='%1.1f%%',
    startangle=90,
    shadow=True,
    labels=None,
    pctdistance=1.12,
    # the ratio between the pie center and start of text label
    explode=explode_list, # 'explode' Lowest 3 continents
)
```

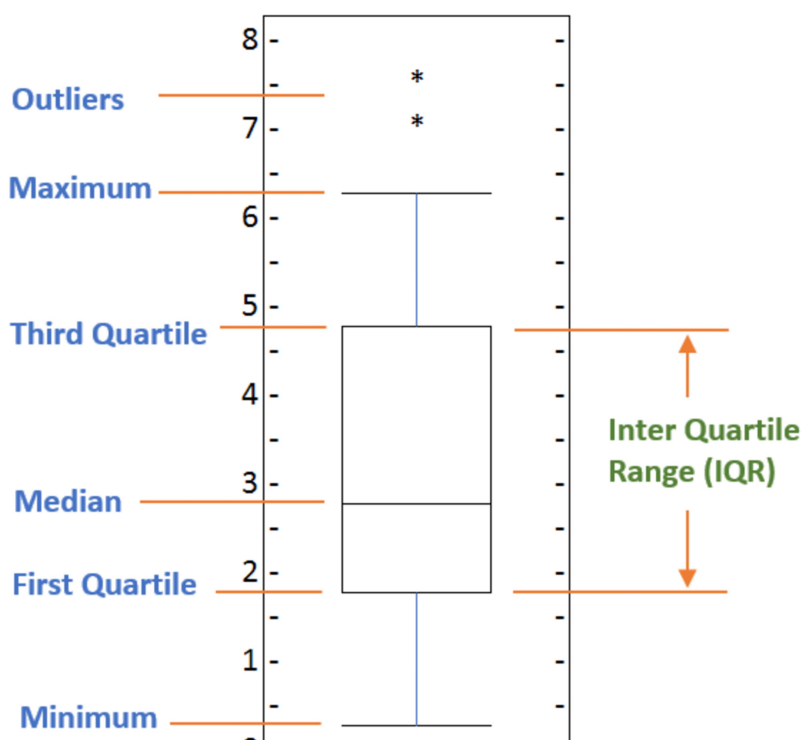
```
plt.title('Immigration to Canada by Continent in 2013', y=1.12)
plt.axis('equal')
plt.legend(labels=df_continents.index, loc='upper left')
plt.show()
```



Result: A pie chart for 2013 with the smallest continents (Oceania, Northern America, and Latin America & the Caribbean) exploded outward for emphasis.

4. Box Plots

A **box plot** summarizes a dataset's distribution through five key statistics: **minimum**, **first quartile (Q1)**, **median (Q2)**, **third quartile (Q3)**, and **maximum**. It's **excellent for spotting outliers** and comparing distributions side by side.



U[- _____]

Box Plot for Japanese Immigration (1980–2013)

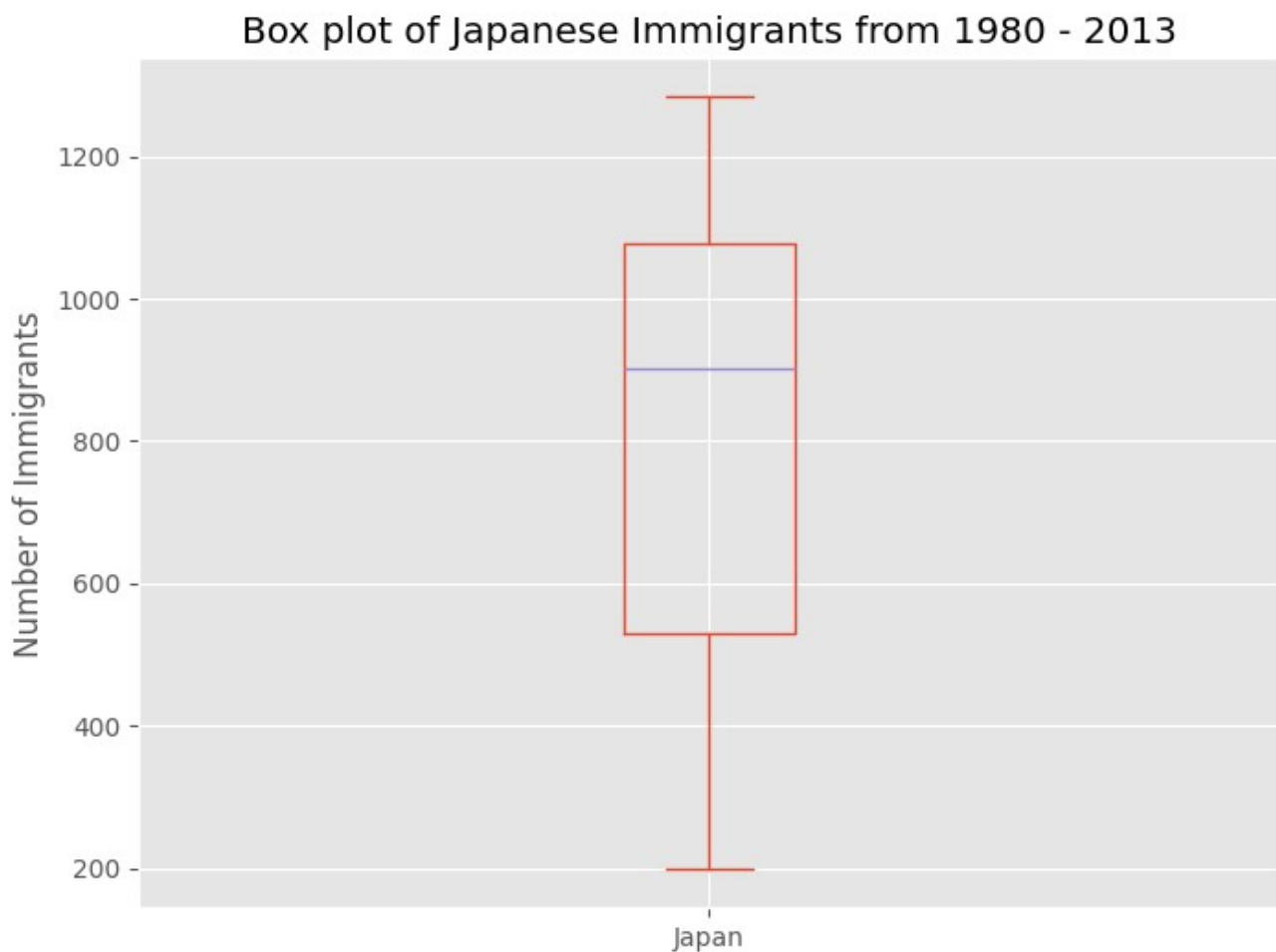
Step 1: Extract Japan's data as a transposed dataframe.

```
df_japan = df_can.loc[['Japan'], years].transpose()  
df_japan.head()
```

Country	Japan
1980	701
1981	756
1982	598
1983	309
1984	246

Step 2: Plot the box plot.

```
df_japan.plot(kind='box', figsize=(8, 6)) # kind='box'  
plt.title('Box plot of Japanese Immigrants from 1980 - 2013')  
plt.ylabel('Number of Immigrants')  
plt.show()
```



Result: A vertical box plot showing:

- **Minimum** $\approx$ 200 immigrants
- **Maximum** $\approx$ 1,300 immigrants
- **Median** $\approx$ 900 immigrants
- **Q1** ($\sim$ 25th percentile) $\approx$ 500 immigrants
- **Q3** ($\sim$ 75th percentile) $\approx$ 1,100 immigrants

```
df_japan.describe()
```

Country	Japan
count	34.000000
mean	814.911765
std	337.219771
min	198.000000
25%	529.000000
50%	902.000000
75%	1079.000000
max	1284.000000

One of the key benefits of box plots is comparing the distribution of multiple datasets. Let's analyze China and India which had very similar immigration trends.

Exercise: Comparing China and India

Goal: Use box plots to compare the distribution of immigrants from China and India across 1980 - 2013.

Step 1: Extract the data for both countries.

```
df_CI = df_can.loc[['China', 'India'], years].transpose()  
df_CI.head()
```

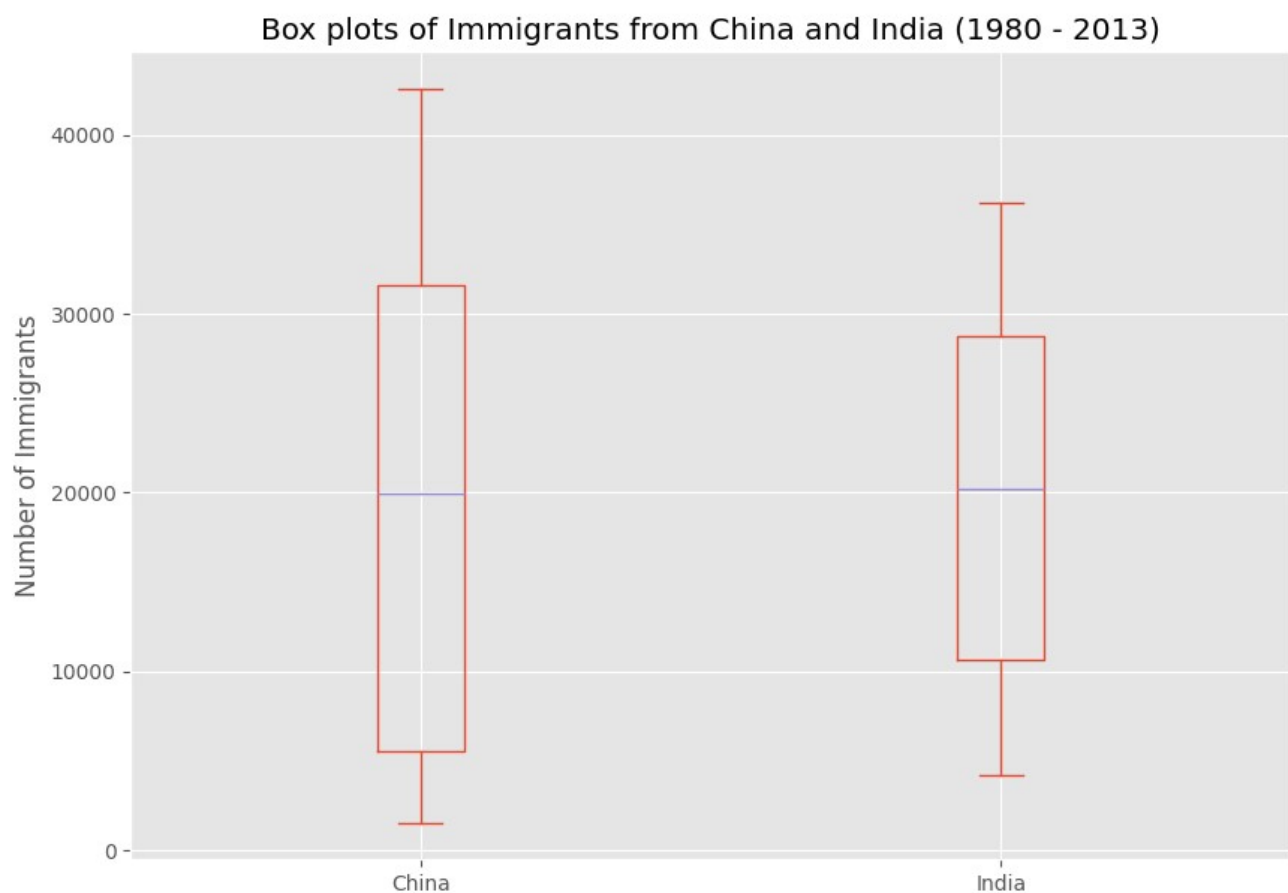
Country	China	India
1980	5123	8880
1981	6682	8670
1982	3308	8147
1983	1863	7338
1984	1527	5704

```
df_CI.describe()
```

Country	China	India
count	34.000000	34.000000
mean	19410.647059	20350.117647
std	13568.230790	10007.342579
min	1527.000000	4211.000000
25%	5512.750000	10637.750000
50%	19945.000000	20235.000000
75%	31568.500000	28699.500000
max	42584.000000	36210.000000

Step 2: Plot the box plots.

```
df_CI.plot(kind='box', figsize=(10, 7))
plt.title('Box plots of Immigrants from China and India (1980 - 2013)')
plt.ylabel('Number of Immigrants')
plt.show()
```

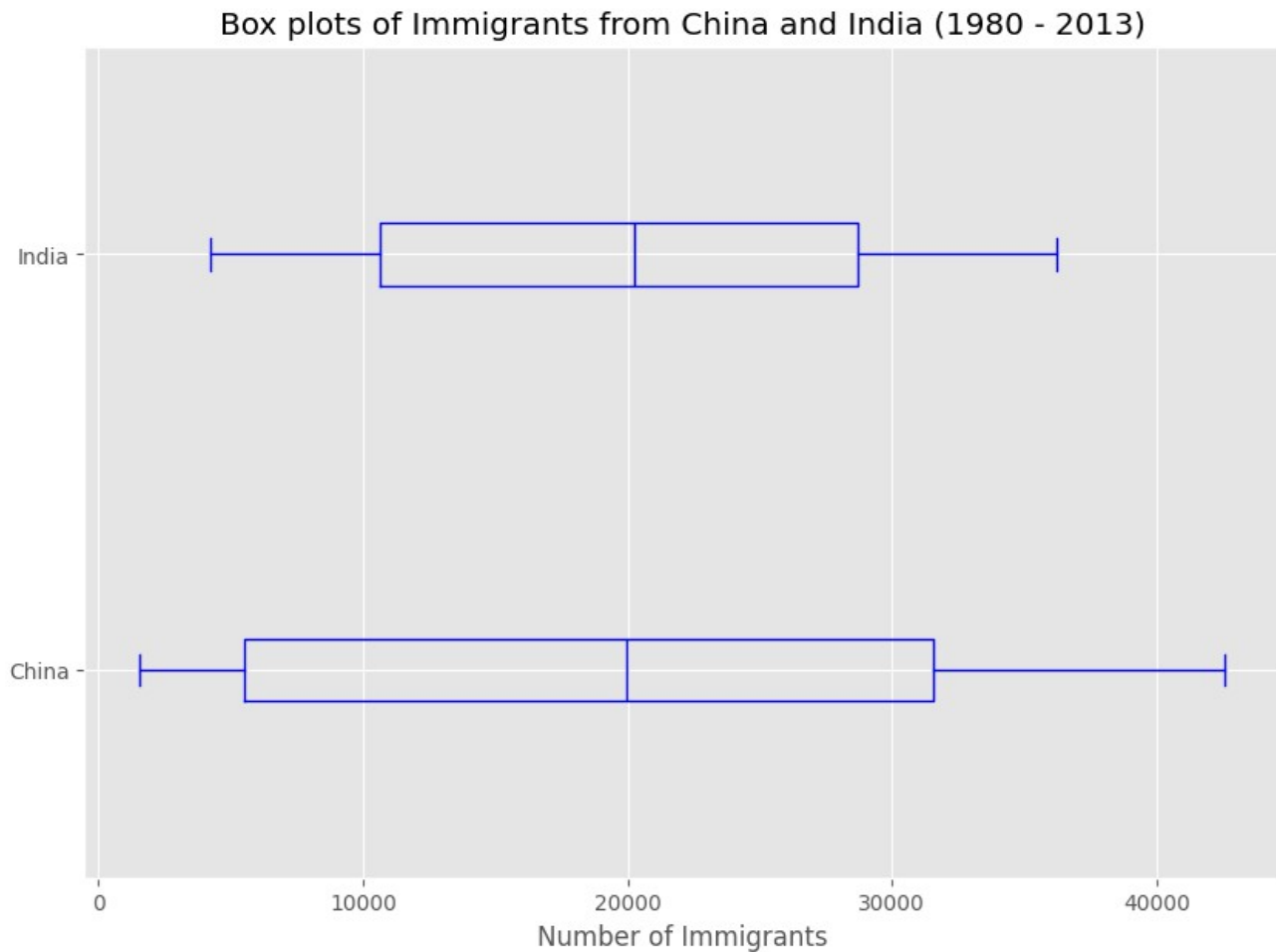


Result: Both countries have a similar median ($\approx 20,000$), but China's range is wider. China's maximum (42,584) is about 15% higher than India's (36,210).

For a horizontal view: (set **vert=False**)

```
df_CI.plot(kind='box', figsize=(10, 7), color='blue', vert=False)
plt.title('Box plots of Immigrants from China and India (1980 - 2013)')
```

```
plt.xlabel('Number of Immigrants')
plt.show()
```

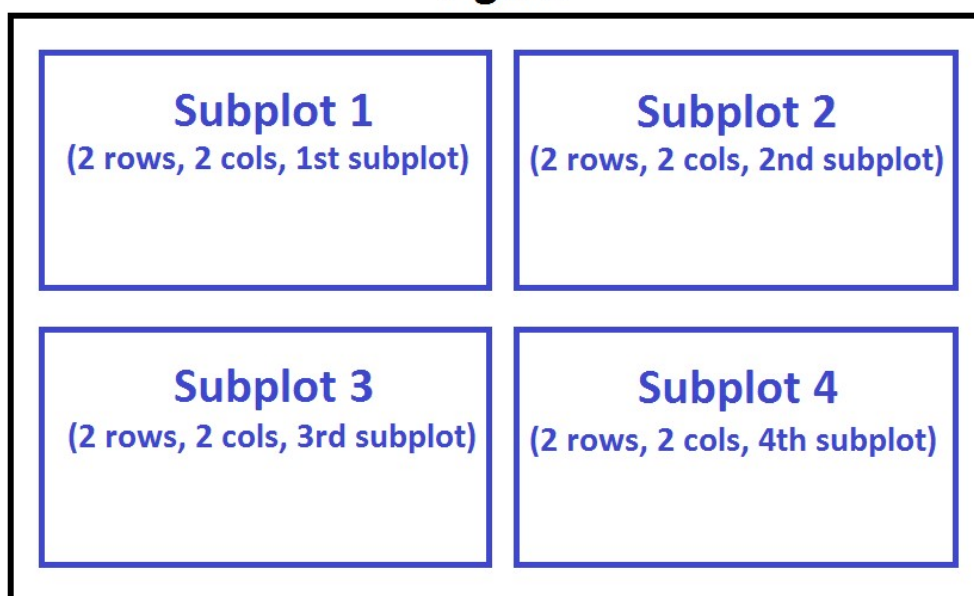


Result: The same comparison flipped horizontally - useful when country names are long.

Subplots: Box Plot + Line Plot Side by Side

Subplots let us compare **multiple visualizations** in one figure. **Artist layer (ax.)** is usually used with **subplots**.

Figure



```

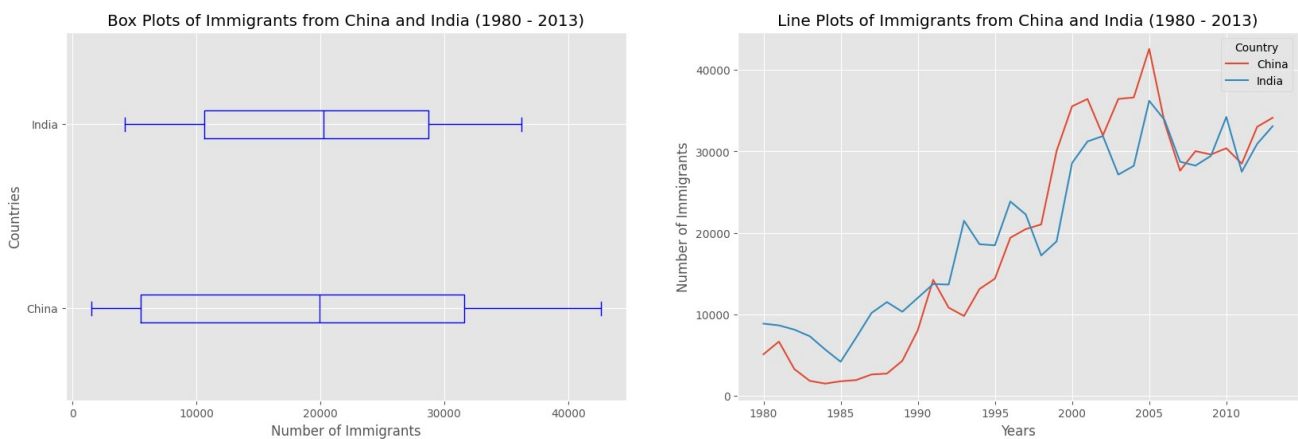
fig = plt.figure() # create figure

# ax = fig.add_subplot(nrows, ncols, plot_number)
ax0 = fig.add_subplot(1, 2, 1) # Tip: subplot(211) == subplot(2, 1, 1)
ax1 = fig.add_subplot(1, 2, 2)

df_CI.plot(kind='box', color='blue', vert=False, figsize=(20, 6), ax=ax0)
ax0.set_title('Box Plots of Immigrants from China and India (1980 - 2013)')
ax0.set_xlabel('Number of Immigrants')
ax0.set_ylabel('Countries')

df_CI.plot(kind='line', figsize=(20, 6), ax=ax1)
ax1.set_title('Line Plots of Immigrants from China and India (1980 - 2013)')
ax1.set_ylabel('Number of Immigrants')
ax1.set_xlabel('Years')
plt.show()

```



Result: A two-panel figure. The left panel shows box plots; the right panel shows the same data as time-series lines. The line plot reveals that China overtook India around the year 2000, and both show upward trends.

Exercise: Top 15 Countries by Decade

Goal: Create box plots comparing the top 15 countries' immigration totals across the 1980s, 1990s, and 2000s.

Step 1: Get the top 15 countries by total immigration.

```

df_top15 = df_can.sort_values(['Total'], ascending=False, axis=0).head(15)
df_top15.head()

```

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...
Country											
India	Asia	Southern Asia	Developing regions	8880	8670	8147	7338	5704	4211	7150	...
China	Asia	Eastern Asia	Developing regions	5123	6682	3308	1863	1527	1816	1960	...
United Kingdom of Great Britain and Northern Ireland	Europe	Northern Europe	Developed regions	22045	24796	20620	10015	10170	9564	9470	...
Philippines	Asia	South-Eastern Asia	Developing regions	6051	5921	5249	4562	3801	3150	4166	...
Pakistan	Asia	Southern Asia	Developing regions	978	972	1201	900	668	514	691	...

5 rows × 38 columns

Step 2: Aggregate totals by decade.

```
# create a list of all years in decades 80's, 90's, and 00's
years_80s = list(map(str, range(1980, 1990)))
years_90s = list(map(str, range(1990, 2000)))
years_00s = list(map(str, range(2000, 2010)))

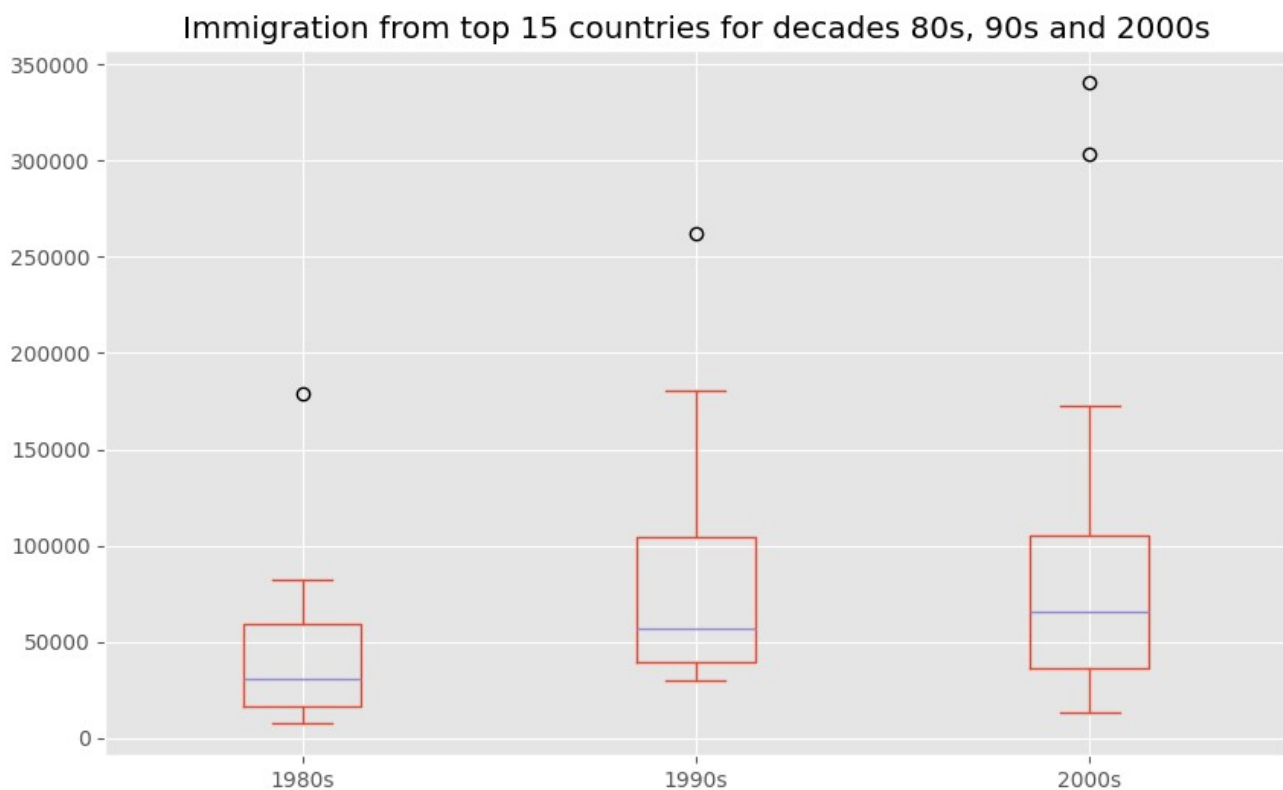
# create a series for each decade
df_80s = df_top15.loc[:, years_80s].sum(axis=1)
df_90s = df_top15.loc[:, years_90s].sum(axis=1)
df_00s = df_top15.loc[:, years_00s].sum(axis=1)

# merge the three series into a new data frame
new_df = pd.DataFrame({'1980s': df_80s, '1990s': df_90s, '2000s': df_00s})
new_df.head()
```

	1980s	1990s	2000s
Country			
India	82154	180395	303591
China	32003	161528	340385
United Kingdom of Great Britain and Northern Ireland	179171	261966	83413
Philippines	60764	138482	172904
Pakistan	10591	65302	127598

Step 3: Plot the box plots.

```
new_df.plot(kind='box', figsize=(10, 6))
plt.title('Immigration from top 15 countries for decades 80s, 90s and 2000s')
plt.show()
```



```
new_df.describe()
```

	1980s	1990s	2000s
count	15.000000	15.000000	15.000000
mean	44418.333333	85594.666667	97471.533333
std	44190.676455	68237.560246	100583.204205
min	7613.000000	30028.000000	13629.000000
25%	16698.000000	39259.000000	36101.500000
50%	30638.000000	56915.000000	65794.000000
75%	59183.000000	104451.500000	105505.500000
max	179171.000000	261966.000000	340385.000000

The box plot differs from the summary table created. The box plot scans the data and identifies the outliers. In order to be an outlier, the data value must be:

- larger than Q3 by at least 1.5 times the interquartile range (IQR), or,
- smaller than Q1 by at least 1.5 times the IQR.

Let's look at decade 2000s as an example:

- Q1 (25%) = 36,101.5

- $Q3 (75\%) = 105,505.5$
- $IQR = Q3 - Q1 = 69,404$

Using the definition of outlier, any value that is greater than Q3 by 1.5 times IQR will be flagged as outlier.

Outlier > $105,505.5 + (1.5 * 69,404)$

Outlier > 209,611.5

Let's confirm:

```
new_df = new_df.reset_index()
new_df[new_df['2000s'] > 209611.5]
```

	Country	1980s	1990s	2000s
0	India	82154	180395	303591
1	China	32003	161528	340385

China and India stand out because their immigration totals are **disproportionately large** (exceeds 209,611.5 for the decade) compared to the rest of the dataset.

The box plot is an advanced visualization tool, and there are many options and customizations that exceed the scope of this lab. Please refer to [Matplotlib documentation](#) on box plots for more information.

5. Scatter Plots

A scatter plot **maps two variables on a 2D graph** without connecting the points. It's **perfect for spotting trends and relationships**.

Let's visualize total immigration to Canada across all countries for each year from 1980 to 2013.

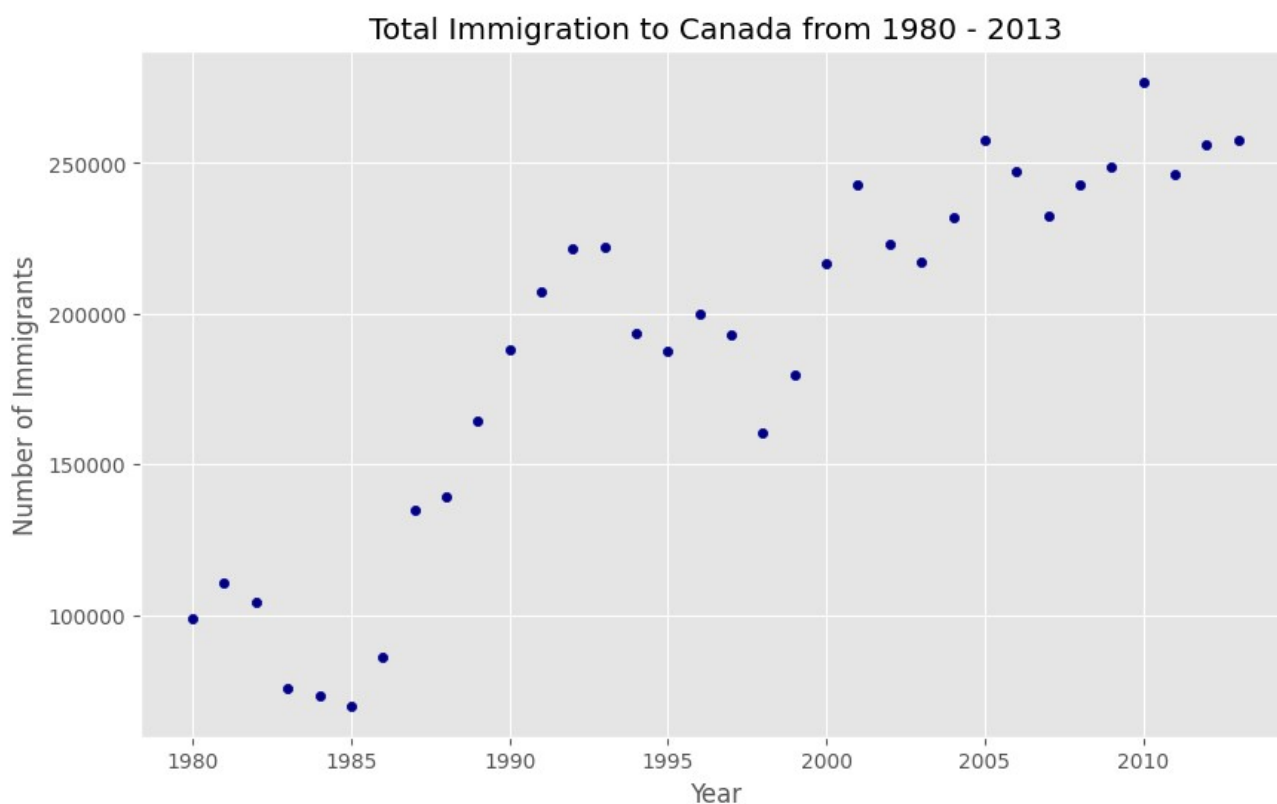
Step 1: Get the total population per year.

```
df_tot = pd.DataFrame(df_can[years].sum(axis=0)) # total population
df_tot.index = map(int, df_tot.index) # for regression later on
df_tot.reset_index(inplace=True)
df_tot.columns = ['year', 'total']
df_tot.head()
```

	year	total
0	1980	99137
1	1981	110563
2	1982	104271
3	1983	75550
4	1984	73417

Step 2: Generate the scatter plot.

```
df_tot.plot(kind='scatter', x='year', y='total', figsize=(10, 6),
color='darkblue')
plt.title('Total Immigration to Canada from 1980 - 2013')
plt.xlabel('Year')
plt.ylabel('Number of Immigrants')
plt.show()
```



Result: A clear upward trend - immigration to Canada has increased steadily over the decades.

Adding a Regression Line

Then fit a **linear regression line** to quantify the trend using Numpy's **polyfit()**

```
x = df_tot['year']
y = df_tot['total']
#Degree of fitting polynomial. 1 = linear, 2 = quadratic, and so on.
fit = np.polyfit(x, y, deg=1)
fit
```

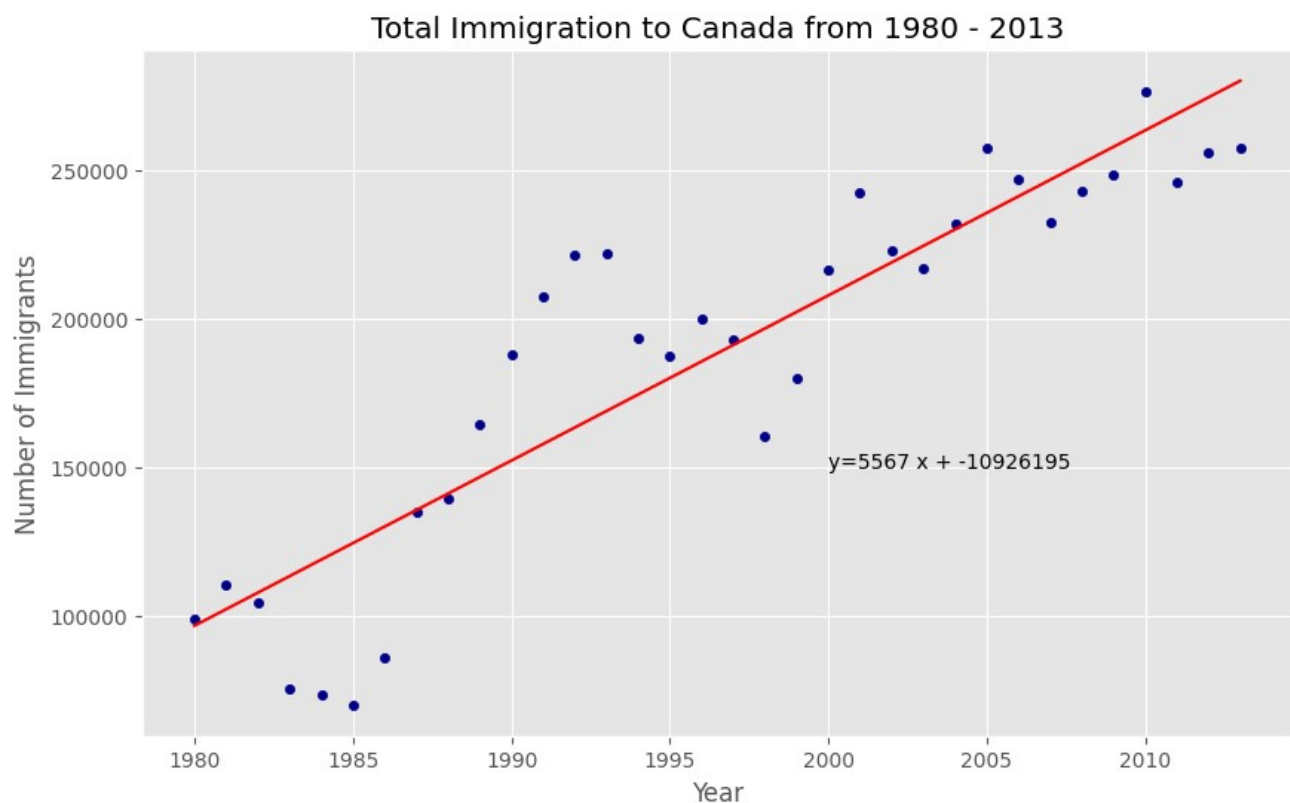


```
array([ 5.56709228e+03, -1.09261952e+07])
```

The **slope** is ~**5,567** and the **intercept** is ~ **-10,926,195**. This gives the equation: **Immigrants = 5,567 × Year – 10,926,195**.

```
df_tot.plot(kind='scatter', x='year', y='total', figsize=(10, 6),
color='darkblue')
plt.title('Total Immigration to Canada from 1980 - 2013')
plt.xlabel('Year')
plt.ylabel('Number of Immigrants')
plt.plot(x, fit[0] * x + fit[1], color='red')
plt.annotate('y={0:.0f} x + {1:.0f}'.format(fit[0], fit[1]), xy=(2000, 150000))
plt.show()
```

```
'No. Immigrants = {0:.0f} * Year + {1:.0f}'.format(fit[0], fit[1])
```



```
'No. Immigrants = 5567 * Year + -10926195'
```

Result: Red regression line across data points. Immigration **took a dip around 1993 - 1997** due to **Bill C-86** making it more difficult for family members (other than nuclear family) to immigrate to Canada.

Using the equation of line of best fit, the number of immigrants in 2015 can be estimated:

```
No. Immigrants = 5567 * Year - 10926195
No. Immigrants = 5567 * 2015 - 10926195
No. Immigrants = 291,310
```

When compared to the actual from Citizenship and Immigration Canada's (CIC) [2016 Annual Report](#), Canada accepted **271,845** immigrants in 2015, **7% off from the estimation** - a solid estimate given the data source.

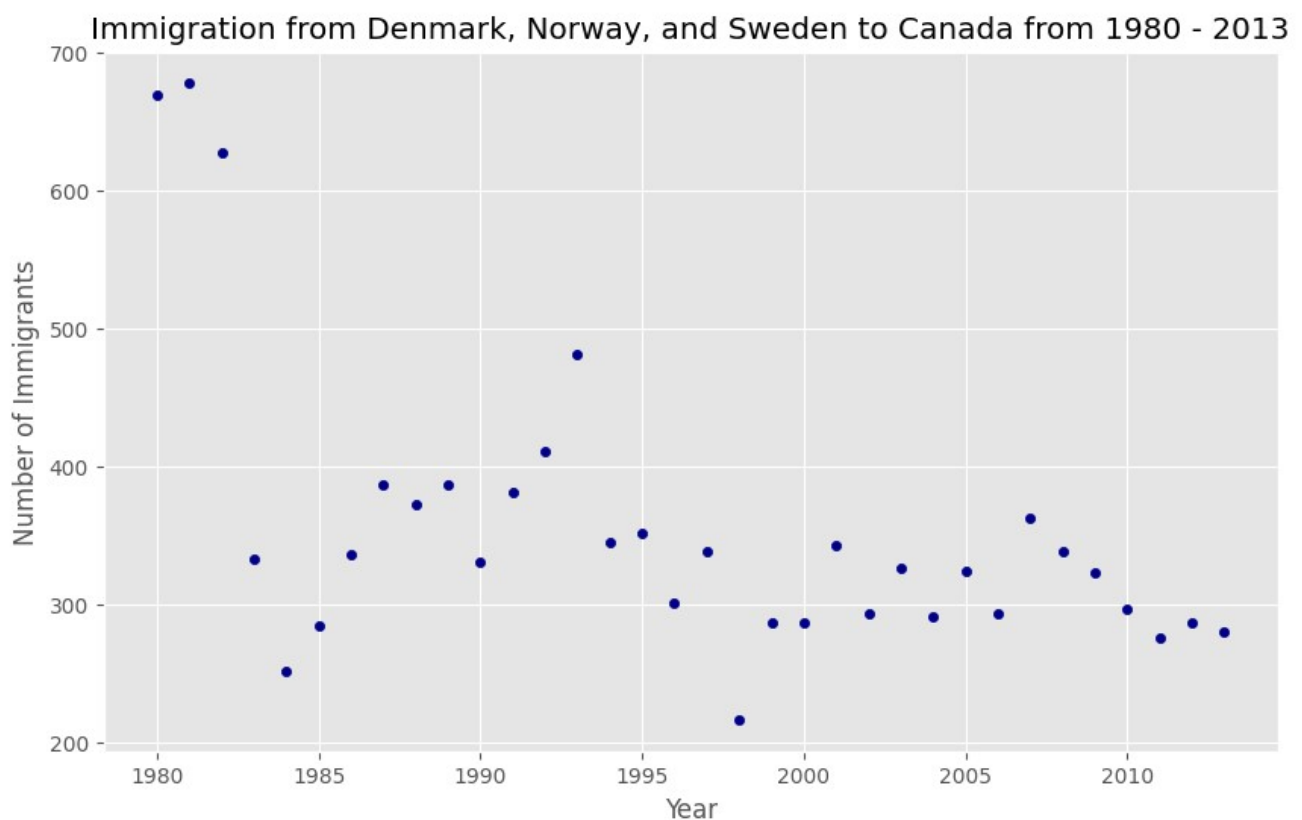
Exercise: Denmark, Norway, and Sweden

Goal: Create a scatter plot showing total immigration from Denmark, Norway, and Sweden combined.

```
df_countries = df_can.loc[['Denmark', 'Norway', 'Sweden'], years].transpose()
df_total = pd.DataFrame(df_countries.sum(axis=1))
df_total.reset_index(inplace=True)
df_total.columns = ['year', 'total']
df_total['year'] = df_total['year'].astype(int)
df_total.head()
```

	year	total
0	1980	669
1	1981	678
2	1982	627
3	1983	333
4	1984	252

```
df_total.plot(kind='scatter', x='year', y='total', figsize=(10, 6),
color='darkblue')
plt.title('Immigration from Denmark, Norway, and Sweden to Canada from 1980 -
2013')
plt.xlabel('Year')
plt.ylabel('Number of Immigrants')
plt.show()
```



Result: A scatter plot showing a generally declining trend. Immigration from these Scandinavian countries has decreased over time.

6. Bubble Plots

A **bubble plot** extends the scatter plot by adding a third dimension: the **size** of each bubble represents a third variable (the "**weight**").

I'll use this to analyze immigration from **Argentina** and **Brazil** and see how economic crises affected emigration.

Step 1: Prepare a transposed dataframe with years as integers.

```
df_can_t = df_can[years].transpose()
df_can_t.index = map(int, df_can_t.index)
df_can_t.index.name = 'Year'
df_can_t.reset_index(inplace=True)
df_can_t.head()
```

Country	Year	Afghanistan	Albania	Algeria	American Samoa	Andorra	Angola	Antigua and Barbuda	Argentina	A
0	1980	16	1	80	0	0	1	0	368	
1	1981	39	0	67	1	0	3	0	426	
2	1982	39	0	71	0	0	6	0	626	
3	1983	47	0	69	0	0	6	0	241	
4	1984	71	0	63	0	0	4	42	237	

5 rows × 196 columns

Step 2: Normalize the data for bubble size using **min-max scaling** (values 0–1). The general formula is:

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

where X is the original value, X' is the corresponding normalized value. The formula sets the max value in the dataset to 1, and sets the min value to 0. The rest of the data points are scaled to a value between 0-1 accordingly.

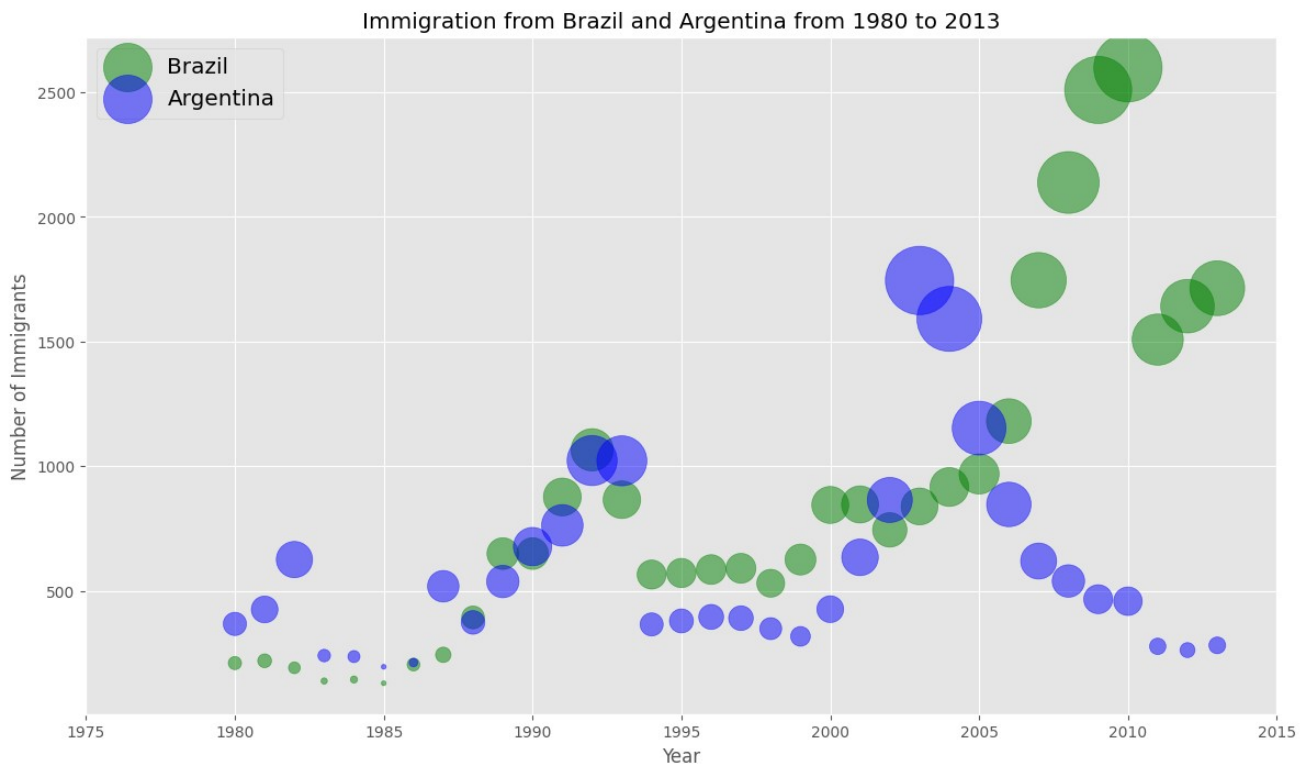
```
norm_brazil = (df_can_t['Brazil'] - df_can_t['Brazil'].min()) /
(df_can_t['Brazil'].max() - df_can_t['Brazil'].min())
norm_argentina = (df_can_t['Argentina'] - df_can_t['Argentina'].min()) /
(df_can_t['Argentina'].max() - df_can_t['Argentina'].min())
```

Step 3: Plot the bubble chart.

```
# Brazil
ax0 = df_can_t.plot(
    kind='scatter',
    x='Year',
    y='Brazil',
    figsize=(14, 8),
    alpha=0.5,
    color='green',
    s=norm_brazil * 2000 + 10, # upscale for visibility and
    xlim=(1975, 2015)
)

# Argentina
ax1 = df_can_t.plot(
    kind='scatter',
    x='Year',
    y='Argentina',
    alpha=0.5,
    color='blue',
    s=norm_argentina * 2000 + 10, # +10 for points with zero weights
    ax=ax0
)

ax0.set_ylabel('Number of Immigrants')
ax0.set_title('Immigration from Brazil and Argentina from 1980 to 2013')
ax0.legend(['Brazil', 'Argentina'], loc='upper left', fontsize='x-large')
plt.show()
```



Result: A bubble plot where bubble **size reflects the relative magnitude of immigration** for each year. **Key observations:**

- Argentina shows a spike in immigration during its **1998–2002 great depression** (larger bubbles).

- Another spike appears around **1985–1993**, corresponding to an earlier economic downturn.
- Brazil shows a surge around **2010**, following the recovery from the Samba Effect currency crisis of 1999.

Exercise: China vs. India Bubble Plot

Goal: Create a bubble plot comparing immigration from China and India.

Step 1: Normalize the data.

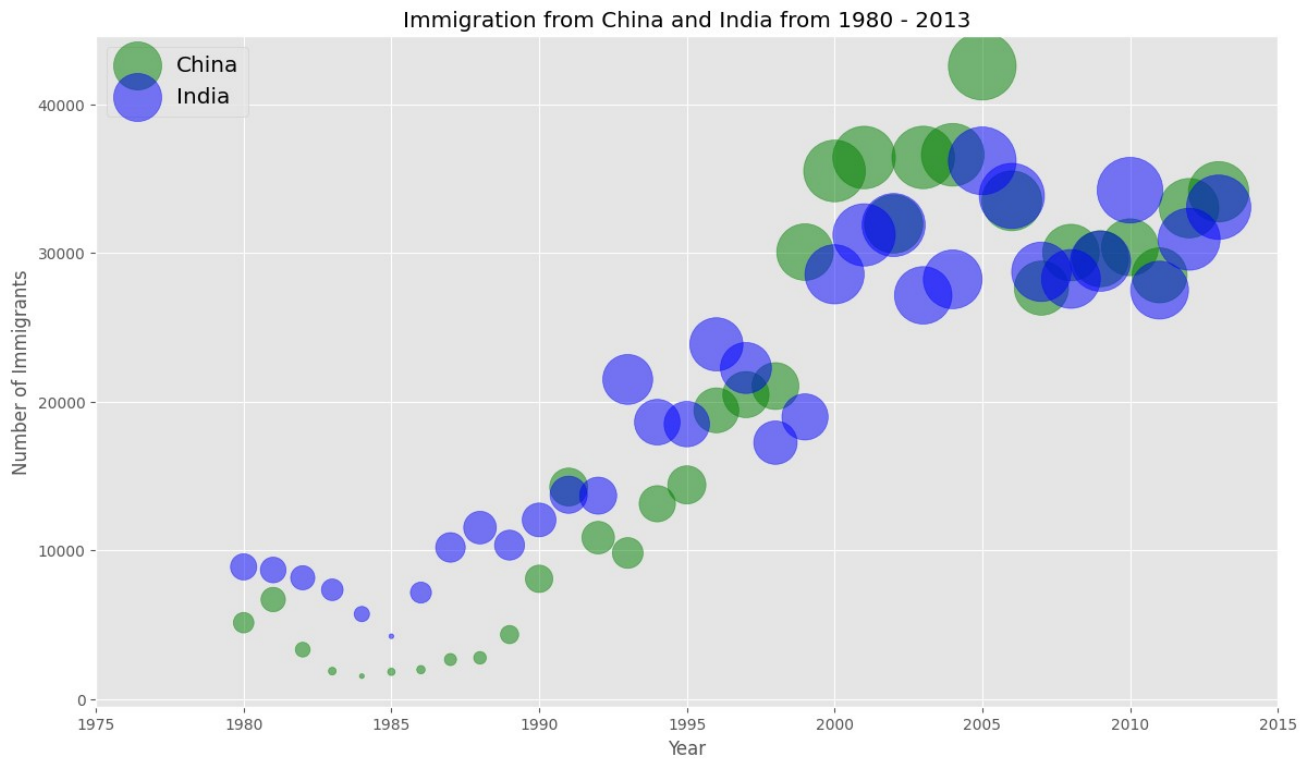
```
norm_china = (df_can_t['China'] - df_can_t['China'].min()) /
(df_can_t['China'].max() - df_can_t['China'].min())
norm_india = (df_can_t['India'] - df_can_t['India'].min()) /
(df_can_t['India'].max() - df_can_t['India'].min())
```

Step 2: Generate the bubble plots.

```
ax0 = df_can_t.plot(
    kind='scatter',
    x='Year',
    y='China',
    figsize=(14, 8),
    alpha=0.5,
    color='green',
    s=norm_china * 2000 + 10,
    xlim=(1975, 2015)
)

ax1 = df_can_t.plot(
    kind='scatter',
    x='Year',
    y='India',
    alpha=0.5,
    color='blue',
    s=norm_india * 2000 + 10,
    ax=ax0
)

ax0.set_ylabel('Number of Immigrants')
ax0.set_title('Immigration from China and India from 1980 - 2013')
ax0.legend(['China', 'India'], loc='upper left', fontsize='x-large')
plt.show()
```



Result: Both countries show growing bubbles over time, with China overtaking India around 2000. Bubble sizes make the trend visible without connecting lines.

That's the full walkthrough. I prepared the dataset, grouped and aggregated with pandas, and built four Matplotlib plots - **pie**, **box**, **scatter**, and **bubble** and **run a regression** forecast.